

Java代码审计基础-Struts2与OGNL

目录

- Struts2的JavaWeb程序需要的核心Java代码
- Java对象如何在HTTP请求中流转
- JSP+Servlet是如何处理Java对象转换的
- Struts2是如何利用OGNL将HTTP字符串转为Java对象的
- 不同版本Struts2对OGNL的支持

额外的：CompoundRootAccessor这个类是多重接口的实现类，例如PropertyAccessor,MethodAccessor,ClassResolver等，是Struts对OGNL的主要扩展

Struts2的绝大多数漏洞，都来自OGNL，而且大部分都是远程命令执行的高危漏洞，所以如果要体系化的研究Struts2漏洞，必须要研究OGNL在Struts2中如何工作的。

我们知道，OGNL是一个独立的模块，不与任何框架相关，Struts2如何与OGNL相结合是要学习的重点，思考几个问题：

- 1、一个简单的由Struts2框架支持的Java Web程序都需要哪些核心代码？
- 2、这中间的HTTP请求数据是如何流转的？
- 3、流转过程中，数据发生了哪些变化？内容变化？类型变化？展现方式的变化？
- 4、传统的JSP+Servlet是如何处理上述问题的？
- 5、引入OGNL之后如何处理上述问题的？

一、Struts2的JavaWeb程序需要的核心Java代码

假如我们在开发一个项目，需求是登录功能，对页面美观、多线程并发没有任何要求，只需要最简单的实现方式和，那么Java相关的代码只需要JSP文件、Java文件即可，例如下面的JSP文件

```
index.jsp x LoginAction.java x User.java x
1 <%@ page language="java" contentType="text/html; charset=UTF-8"
2   pageEncoding="UTF-8"%>
3 <!DOCTYPE html PUBLIC "-//W3C//DTD HTML 4.01 Transitional//EN" "http://www.w3.org/TR/html4/loose.dtd">
4 <html>
5 <head>
6   <meta http-equiv="Content-Type" content="text/html; charset=UTF-8">
7   <title>S2-001</title>
8 </head>
9 <body>
10  <h2>Ognl Demo</h2>
11  <form method="post" action="{pageContext.request.contextPath }/login">
12    <input name="user.username" type="text" />
13    <input name="user.password" type="text" />
14    <input type="submit" value="提交" />
15  </form>
16 </body>
17 </html>
```

Java文件-User.java



```
1 package org.example;|
2
3 3 usages
4 public class User {
5     2 usages
6     private String username;
7     2 usages
8     private String password;
9
10
11     1 usage
12     public String getUsername() {
13         return username;
14     }
15
16
17     no usages
18     public void setUsername(String username) {
19         this.username = username;
20     }
21
22     1 usage
23     public String getPassword() {
24         return password;
25     }
26
27     no usages
28     public void setPassword(String password) {
29         this.password = password;
30     }
31 }

```

The image shows a code editor with three tabs: index.jsp, LoginAction.java, and User.java. The User.java tab is active, displaying the following code with annotations: Line 1: package org.example; Line 2: (empty) Line 3: 3 usages (above the class declaration) Line 4: public class User { Line 5: 2 usages (above the first private field) Line 6: private String username; Line 7: 2 usages (above the second private field) Line 8: private String password; Line 9: (empty) Line 10: (empty) Line 11: 1 usage (above the getUsername method) Line 12: public String getUsername() { Line 13: return username; Line 14: } Line 15: (empty) Line 16: no usages (above the setUsername method) Line 17: public void setUsername(String username) { Line 18: this.username = username; Line 19: } Line 20: (empty) Line 21: 1 usage (above the getPassword method) Line 22: public String getPassword() { Line 23: return password; Line 24: } Line 25: (empty) Line 26: no usages (above the setPassword method) Line 27: public void setPassword(String password) { Line 28: this.password = password; Line 29: } Line 30: } Line 31: }

Java文件-LoginAction

```
Index.jsp x>LoginAction.java x>User.java x
4
5 1 usage
6 public class LoginAction extends ActionSupport {
7     4 usages
8     private User user;
9     public String execute() throws Exception {
10
11         if (user.getUsername().equals("admin") && user.getPassword().equals("admin")) {
12             System.out.println("Login success.");
13             return "success";
14         }
15         return "error";
16     }
17
18     no usages
19     public User getUser() {
20         return user;
21     }
22
23     no usages
24     public void setUser(User user) {
25         this.user = user;
26     }
27 }
```

当然除了JSP文件、Java文件还有其他的Maven、Struts、Web的配置文件，这不是我们说明的重点，通过分析登录的业务，可以想到登录时所需要核心属性是用户名与密码，两者被封装到了User对象中

二、Java对象如何在HTTP请求中流转

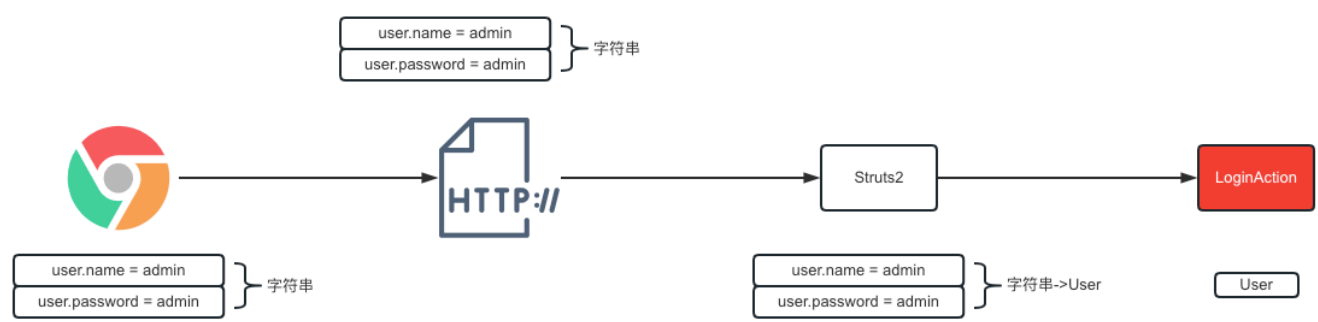
如果按照MVC模型去分析这个项目，对应关系应该是：

Model（数据模型）： User.java

View（视图展示）： index.java

Control（控制器）： LoginAction.java

在想一下数据流转过程



通过上图看看出数据通过Struts2时进行的对象转换，因为HTTP是一个通用协议，与编程语言无关，HTTP可以进行跨编程语言的通信，所以编程语言对象的转换由编程语言自己负责。那我们在抽象下：

View视图的数据与Java对象之间需要一种转换方式，这种方式可能是OGNL，也可能是其它的，这就是表达式引擎。

三、JSP+Servlet是如何处理Java对象转换的？

首先先看下一个最简单的JSP+Servlet的项目是什么样的，我们还做一个登录功能，核心代码如下图

```
m pom.xml (JSPServlet) x Login.java x web.xml x index.jsp x
1 package org.example;
2
3 import javax.servlet.ServletException;
4 import javax.servlet.http.HttpServlet;
5 import javax.servlet.http.HttpServletRequest;
6 import javax.servlet.http.HttpServletResponse;
7 import java.io.IOException;
8
9 1 usage
10 public class Login extends HttpServlet {
11     no usages
12     @Override
13     protected void doPost(HttpServletRequest request, HttpServletResponse response) throws ServletException, IOException {
14         String userName = request.getParameter("username");
15         String password = request.getParameter("password");
16
17         if (userName.equals("admin") && password.equals("admin")){
18             System.out.println("Login success.");
19         }
20     }
21 }
```

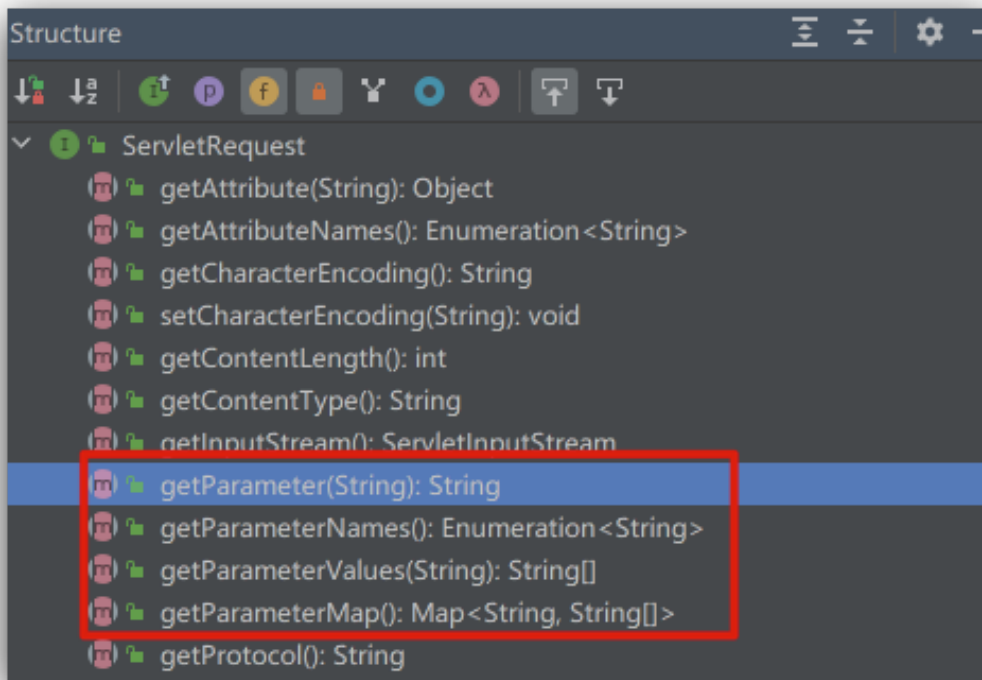
```
m pom.xml (JSPServlet) x Login.java x web.xml x index.jsp x
1 <%@ page language="java" contentType="text/html; charset=UTF-8"
2     pageEncoding="UTF-8"%>
3 <!DOCTYPE html PUBLIC "-//W3C//DTD HTML 4.01 Transitional//EN" "http://www.w3.org/TR/html4/loose.dtd">
4 <html>
5 <head>
6     <meta http-equiv="Content-Type" content="text/html; charset=UTF-8">
7     <title>OGNLTest</title>
8 </head>
9 <body>
10     <h2>Ognl Demo</h2>
11     <form method="post" action="login">
12         <input name="username" type="text" />
13         <input name="password" type="text" />
14         <input type="submit" value="提交" />
15     </form>
16 </body>
17 </html>
```

```
m pom.xml (JSPServlet) x Login.java x web.xml x index.jsp x
1 <!DOCTYPE web-app PUBLIC
2   "-//Sun Microsystems, Inc.//DTD Web Application 2.3//EN"
3   "http://java.sun.com/dtd/web-app_2_3.dtd" >
4
5 <web-app>
6   <display-name>Archetype Created Web Application</display-name>
7   <servlet>
8     <servlet-name>login</servlet-name>
9     <servlet-class>org.example.Login</servlet-class>
10  </servlet>
11  <servlet-mapping>
12    <servlet-name>login</servlet-name>
13    <url-pattern>/login</url-pattern>
14  </servlet-mapping>
15 </web-app>
```

```
m pom.xml (JSPServlet) x Login.java x web.xml x index.jsp x
1 <project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
2   xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/maven-v4_0_0.xsd">
3   <modelVersion>4.0.0</modelVersion>
4   <groupId>org.example</groupId>
5   <artifactId>JSPServlet</artifactId>
6   <packaging>war</packaging>
7   <version>1.0-SNAPSHOT</version>
8   <name>JSPServlet Maven Webapp</name>
9   <url>http://maven.apache.org</url>
10  <dependencies>
11    <dependency>
12      <groupId>junit</groupId>
13      <artifactId>junit</artifactId>
14      <version>3.8.1</version>
15      <scope>test</scope>
16    </dependency>
17    <dependency>
18      <groupId>javax.servlet</groupId>
19      <artifactId>javax.servlet-api</artifactId>
20      <version>3.0.1</version>
21      <scope>provided</scope>
22    </dependency>
23  </dependencies>
24  <build>
25    <finalName>JSPServlet</finalName>
26  </build>
27 </project>
```

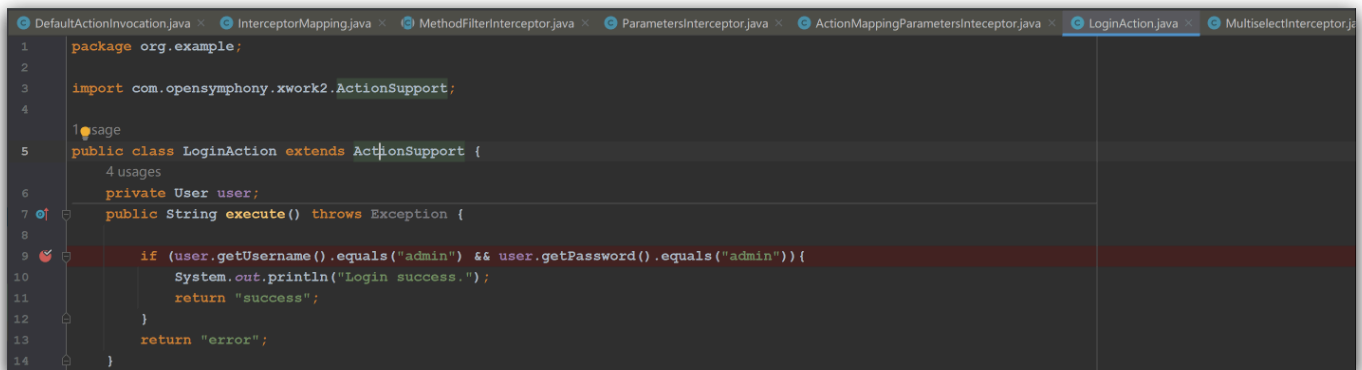
这是最简单的Servlet程序，一丝多余的代码都没有，使用Maven搭建的，这样更方便些。项目包含Login.java、index.jsp、Web.xml。

通过分析Login.java的doPost方法可以发现Servlet是通过HttpServletRequest对象获取HTTP参数，该类的父类ServletRequest类定义了获取HTTP参数的接口，如下图，可以发现返回的参数都是String类型，也就是说Servlet只能接收String类型的参数，想要变成Java对象只能自己转。

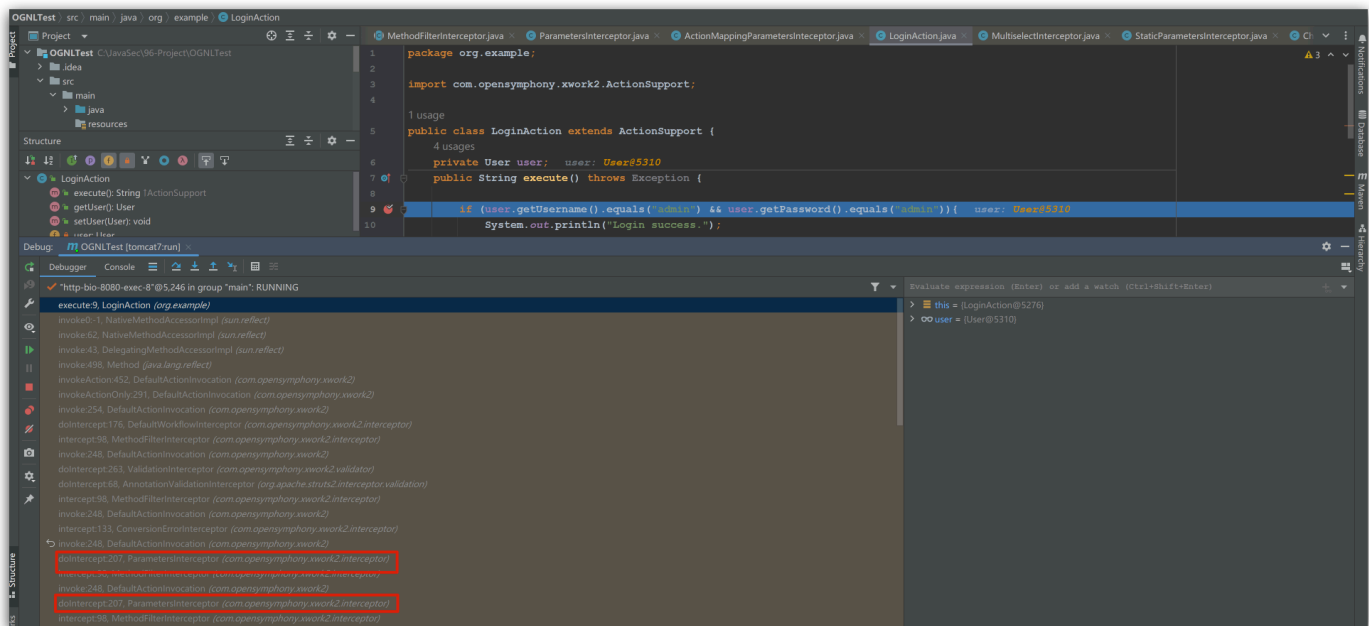


四、Struts2是如何利用OGNL将HTTP字符串转为Java对象的？

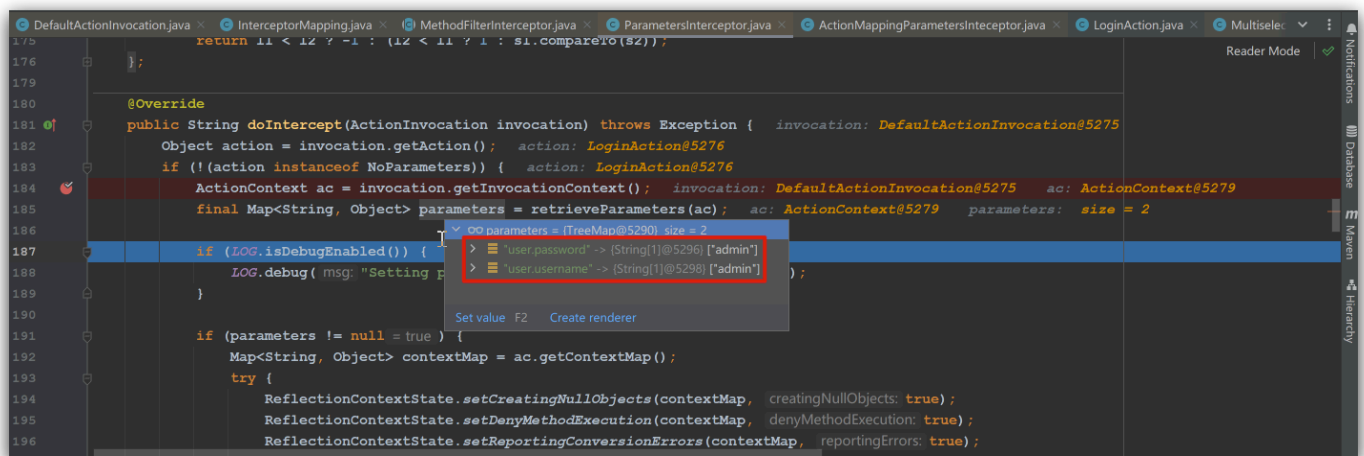
在回到文章最开始利用Struts2搭建的JavaWeb项目，我给它起了一个名字叫做OGNLTest，如果第一次分析Struts2的项目，可以直接在LoginAction中打断点，如下图



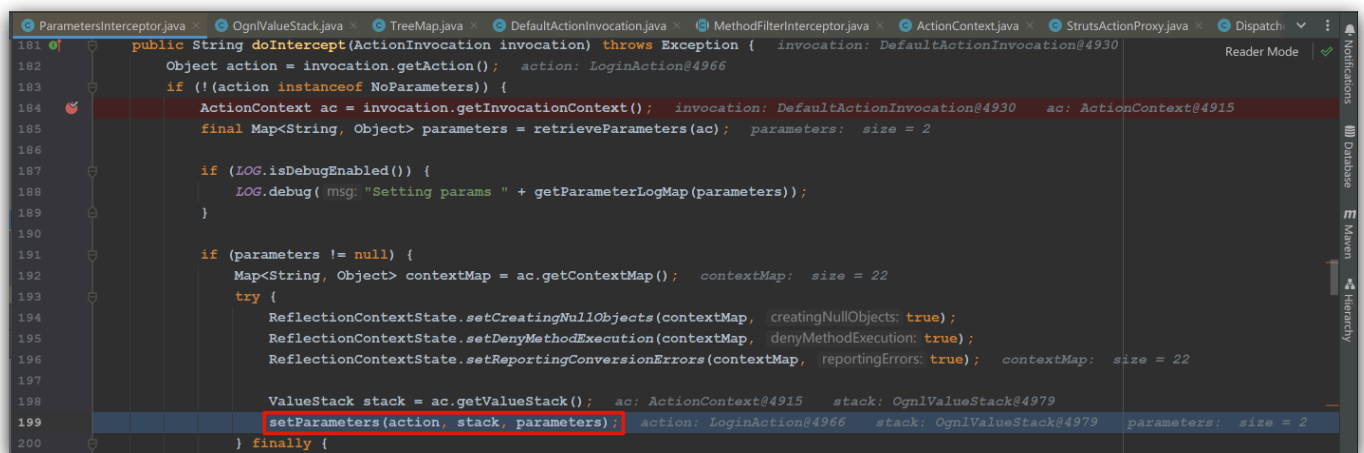
当在浏览器访问URL，输入用户名密码提交后，断点便会在此处停下，此时我们可以看到整个程序的调用栈，如下图，包括针对参数处理的ParametersInterceptor，该类是获取HTTP的参数将其放入ValueStack



在下图位置加断点，可以看到在ParametersInterceptor中获取了参数

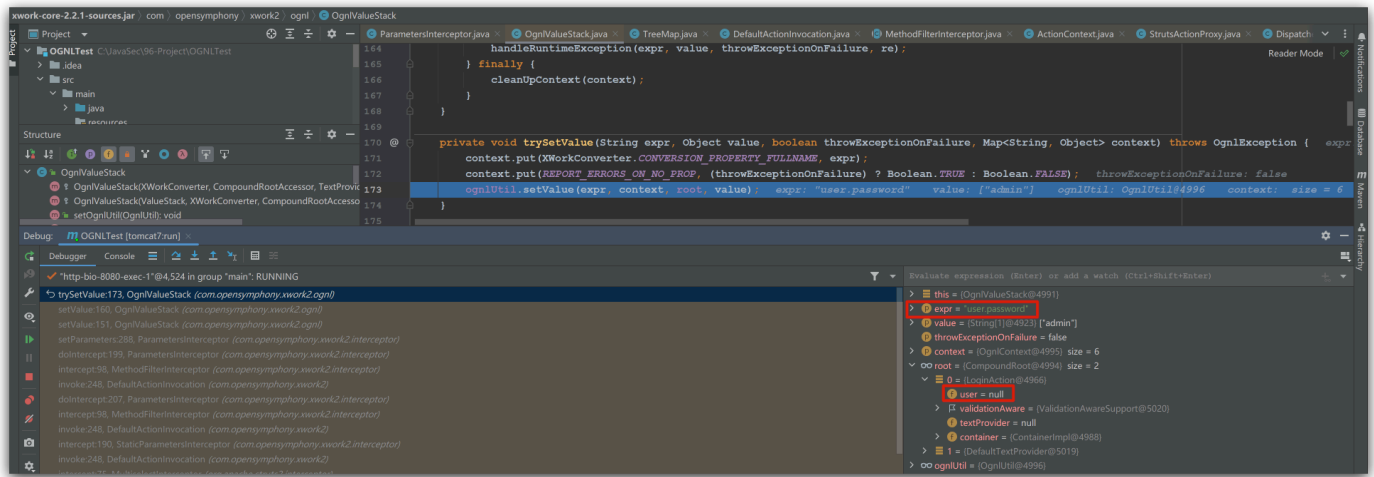


在接下来的代码中，将参数和值放入ValueStack中



ValueStack是有计算功能的，会找到对应的对象，通过OGNL表达式修改Java对象的值，也就是修改LoginAction中的User对象的属性，通过代码调试最终会进入到下图代码

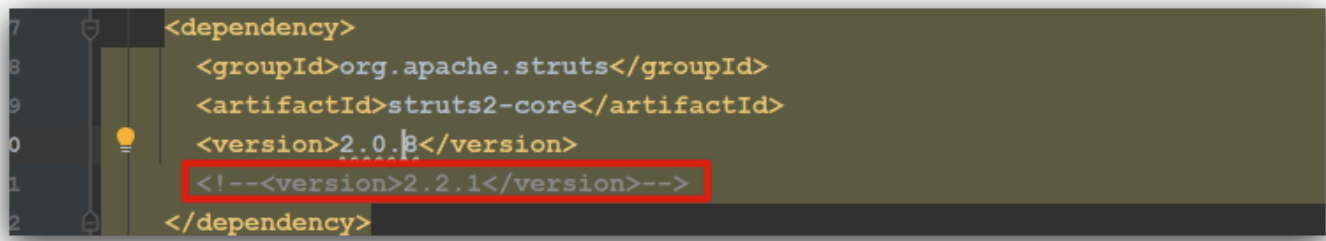
ognlUtil.setValue会根据OGNL的表达式修改root对象中符合表达式规则的对象值，这就是Struts2通过OGNL将HTTP字符串映射到Java对象属性的方法。



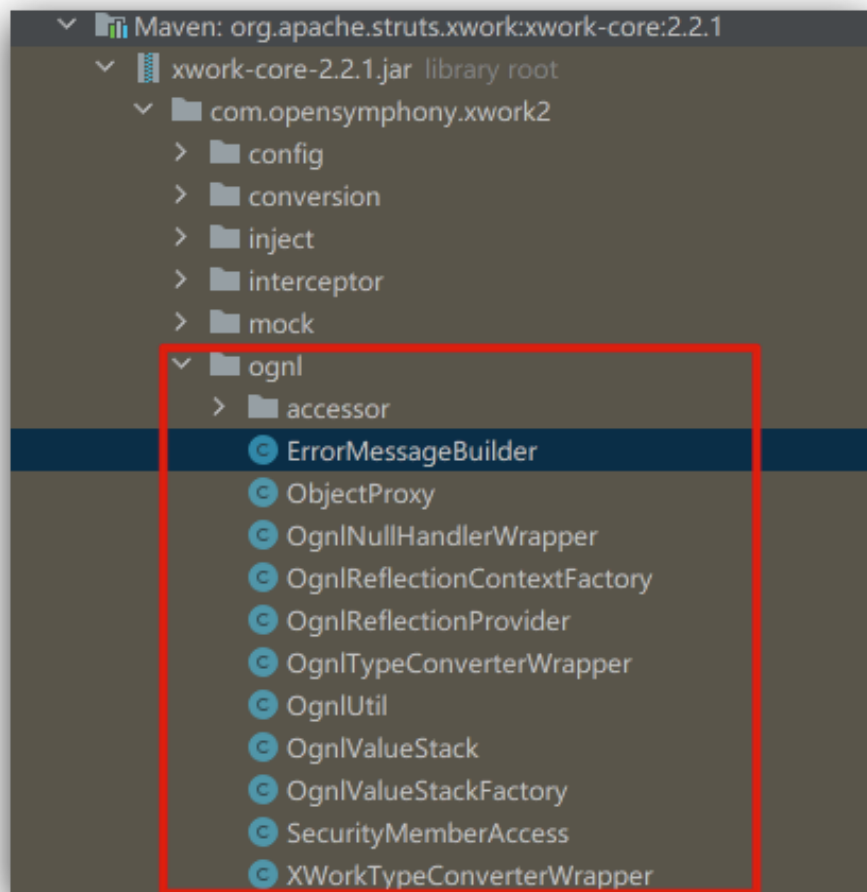
五、不同版本Struts2对OGNL的支持

Struts2对于OGNL的支持发生过很多改变，我将从2个比较大差异的版本对比说起

首先说下2.2.1版本，可以通过修改Struts2-001的Maven漏洞环境的pom.xml，将Struts2的版本改为2.2.1



看下xwork库中的ognl相关类



首先要说的是OgnlValueStack类，该类是对于ValueStack的扩展，增加了对OGNL的计算能力，所以OgnlValueStack具有：**数据存储+ONGL计算的能力**。

是首先看下ValueStack的源码（前半部分省略了），通过方法就可以看出是一个非常典型的栈结构，结构并不复杂。

```
m pom.xml x OgnlValueStack.java x ValueStack.java x Reader Mode
    asType – the type to convert the return value to
    Returns: the result of evaluating the expression
107 0↓ public abstract Object findValue(String expr, Class asType);
108 0↓ public abstract Object findValue(String expr, Class asType, boolean throwExceptionOnFailure);
109
    Get the object on the top of the stack without changing the stack.
    Returns: the object on the top.
    See Also: CompoundRoot.peek()
116 0↓ public abstract Object peek();
117
    Get the object on the top of the stack and remove it from the stack.
    Returns: the object on the top of the stack
    See Also: CompoundRoot.pop()
124 0↓ public abstract Object pop();
125
    Put this object onto the top of the stack
    Params: o – the object to be pushed onto the stack
    See Also: CompoundRoot.push(Object)
132 0↓ public abstract void push(Object o);
133
    Sets an object on the stack with the given key so it is retrievable by findValue(String), findValue
    (String, Class)
    Params: key – the key
           o – the object
141 0↓ public abstract void set(String key, Object o);
142
    Get the number of objects in the stack
    Returns: the number of objects in the stack
148 0↓ public abstract int size();
149
150 }
```

继续看下OgnlValueStack类，它的构造方法都主动调用了setRoot方法进行初始化，将根对象compoundRoot封装到了OgnlValueStack内部，这是一个典型的装饰模式的使用。

```
1 // 设置内部变量root，也是context的根对象
2 this.root = compoundRoot;
3 // OGNL需要的MemberAccess类，权限控制
4 this.securityMemberAccess = new SecurityMemberAccess(allowStaticMethodAccess);
5 // 设置OgnlValueStack的上下文对象，其中将root设置为root对象
6 this.context = Ognl.createDefaultContext(this.root, accessor, new
OgnlTypeConverterWrapper(xworkConverter), securityMemberAccess);
7 // 设置上下文对象的valuestack
8 context.put(VALUE_STACK, this);
```

```
pom.xml x OgnlValueStack.java x ValueStack.java x
63 CompoundRoot root; ❶
64 transient Map<String, Object> context;
65 Class defaultType;
66 Map<Object, Object> overrides;
67 transient OgnlUtil ognlUtil;
68 transient SecurityMemberAccess securityMemberAccess;
69
70 private boolean devMode;
71 private boolean logMissingProperties;
72
73 protected OgnlValueStack(XWorkConverter xworkConverter, CompoundRootAccessor accessor, TextProvider prov, boolean allowStaticAccess) {
74     setRoot(xworkConverter, accessor, new CompoundRoot(), allowStaticAccess);
75     push(prov);
76 }
77
78 @
79 protected OgnlValueStack(ValueStack vs, XWorkConverter xworkConverter, CompoundRootAccessor accessor, boolean allowStaticAccess) {
80     setRoot(xworkConverter, accessor, new CompoundRoot(vs.getRoot()), allowStaticAccess);
81 }
82
83 @Inject
84 public void setOgnlUtil(OgnlUtil ognlUtil) { this.ognlUtil = ognlUtil; }
85
86
87 protected void setRoot(XWorkConverter xworkConverter, CompoundRootAccessor accessor, CompoundRoot compoundRoot, ❷
88     boolean allowStaticMethodAccess) {
89     this.root = compoundRoot; ❸
90     this.securityMemberAccess = new SecurityMemberAccess(allowStaticMethodAccess);
91     this.context = Ognl.createDefaultContext(this.root, accessor, new OgnlTypeConverterWrapper(xworkConverter), securityMemberAccess);
92     context.put(VALUE_STACK, this);
93     Ognl.setClassResolver(context, accessor);
94     ((OgnlContext) context).setTraceEvaluations(false);
95     ((OgnlContext) context).setKeepLastEvaluation(false);
96 }
```

看一下CompoundRoot类，就是一个ArrayList，然后内部模拟栈空间，增加了peek、pop、push这些栈操作的方法，总结来说

OgnlValueStack类实现了数据存储和OGNL计算功能，但本质上是CompoundRoot实现了数据存储的功能，Ognl实现了计算功能

```
pom.xml x OgnlValueStack.java x CompoundRoot.java x ValueStack.java x
1 ../../
16 package com.opensymphony.xwork2.util;
17
18 import ...
19
20
21
22 A Stack that is implemented using a List.
23 Author: plightbo
24 Version: $Revision: 894090 $
25
26
27 public class CompoundRoot extends ArrayList {
28
29     public CompoundRoot() {
30     }
31
32
33 @
34     public CompoundRoot(List list) { super(list); }
35
36
37
38     public CompoundRoot cutStack(int index) { return new CompoundRoot(subList(index, size())); }
39
40
41     public Object peek() { return get(0); }
42
43
44     public Object pop() { return remove(index: 0); }
45
46
47     public void push(Object o) { add(index: 0, o); }
48
49
50
51
52
53 }
```

ValueStack如何取值呢？

下图是ValueStack的数据结构，当针对ValueStack进行OGNL表达式计算时，会得到下面的结果

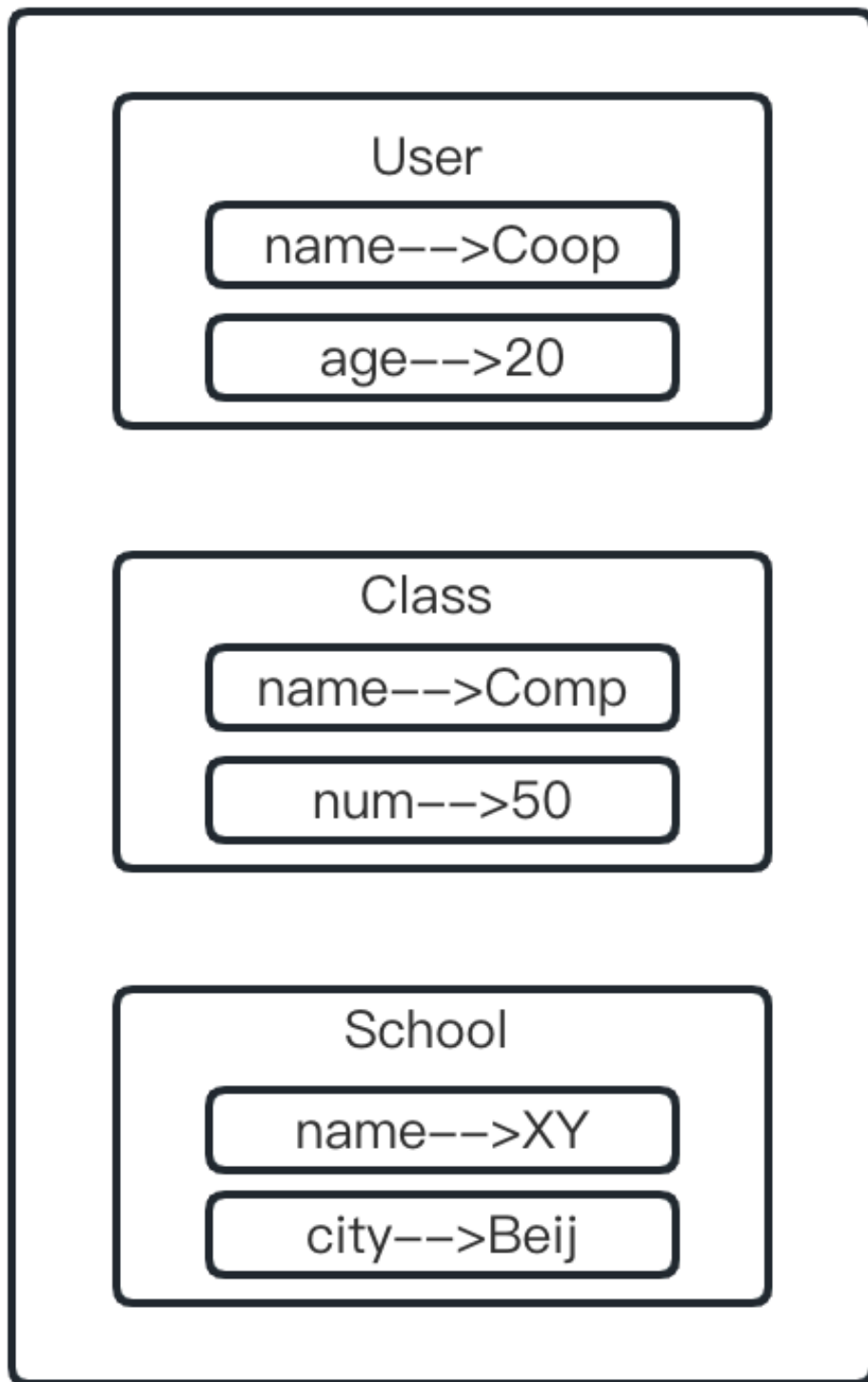
1、调用user.getAge()会返回18

2、调用school.ctiy会返回Beij

可以看出位于ValueStack中的对象，一律视为OGNL的ROOT对象与栈顶还是栈底的位置没有关系

OGNL表达式计算的逻辑是：从栈顶开始匹配，返回第一个匹配到的表达式计算结果

栈顶



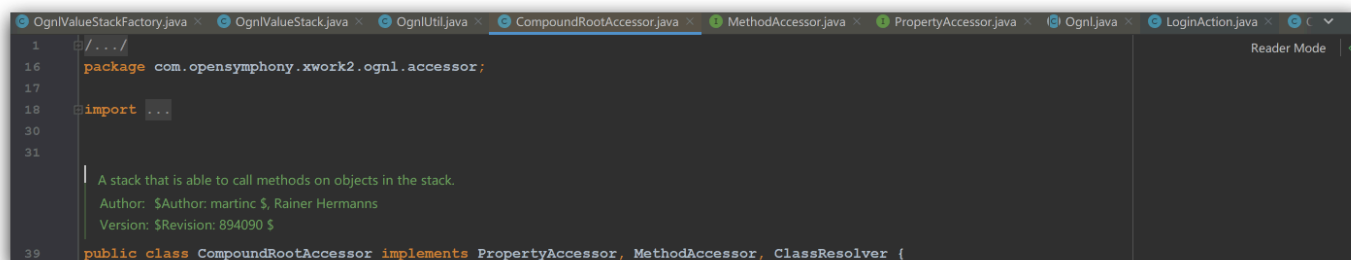
查找方向

栈底

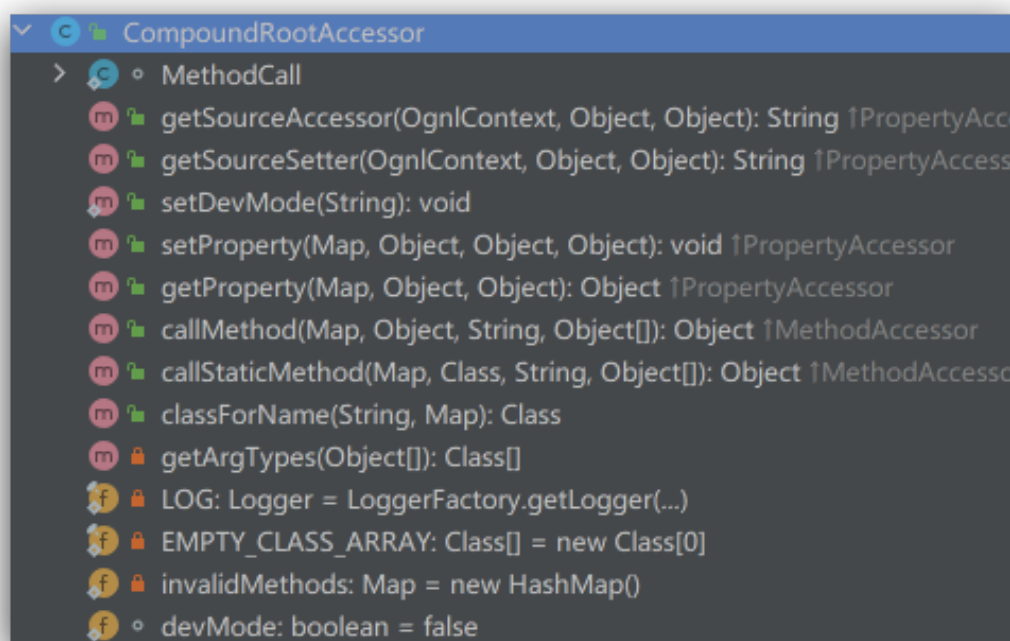
通过OGNL表达式访问ValueStack（本质是获取CompoundRoot的元素）的元素，是通过CompoundRootAccessor来实现的

CompoundRootAccessor实现了对CompoundRoot中元素以表达式方式的访问，

CompoundRootAccessor类扩展了ognl的PropertyAccessor,MethodAccessor,ClassResolver



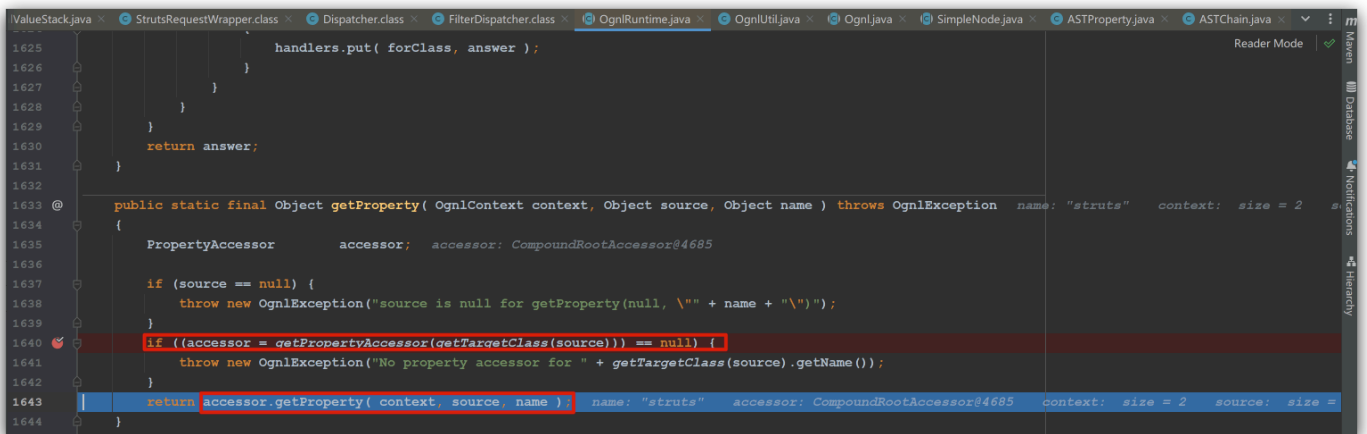
```
1 // ...
16 package com.opensymphony.xwork2.ognl.accessor;
17
18 import ...
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39 public class CompoundRootAccessor implements PropertyAccessor, MethodAccessor, ClassResolver {
```



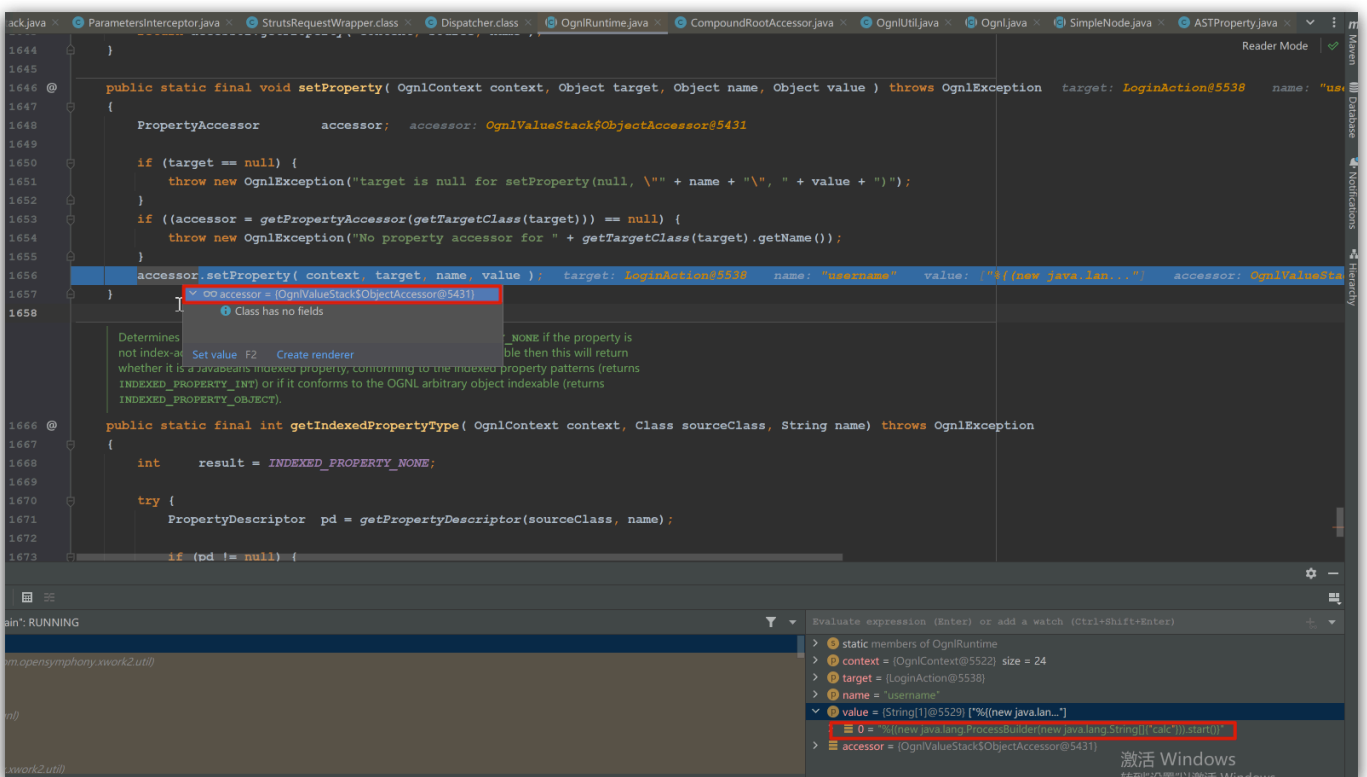
总结下讲到的类：

- OgnlValueStackFactory
 - 被Inject标记的setContainer在框架初始化时执行，为后续Ognl解析OgnlValueStack内的值配置PropertyAccessor、MethodAccessor、NullHandler
 - 指定OgnlValueStack内部CompoundRoot的访问类CompoundRootAccessor
- OgnlValueStack
 - 当执行OgnlValueStackFactory.createValueStack时开始初始化，构造方法内主动调用setRoot方法
 - setRoot方法内指定了CompoundRoot、OgnlValueStack的context、classResovler
- CompoundRoot
 - 就是一个ArrayList，用于存储OgnlValueStack内真正要存储的对象
- CompoundRootAccessor
 - 提供了一些列的接口，支持OGNL表达式操作CompoundRoot内的对象

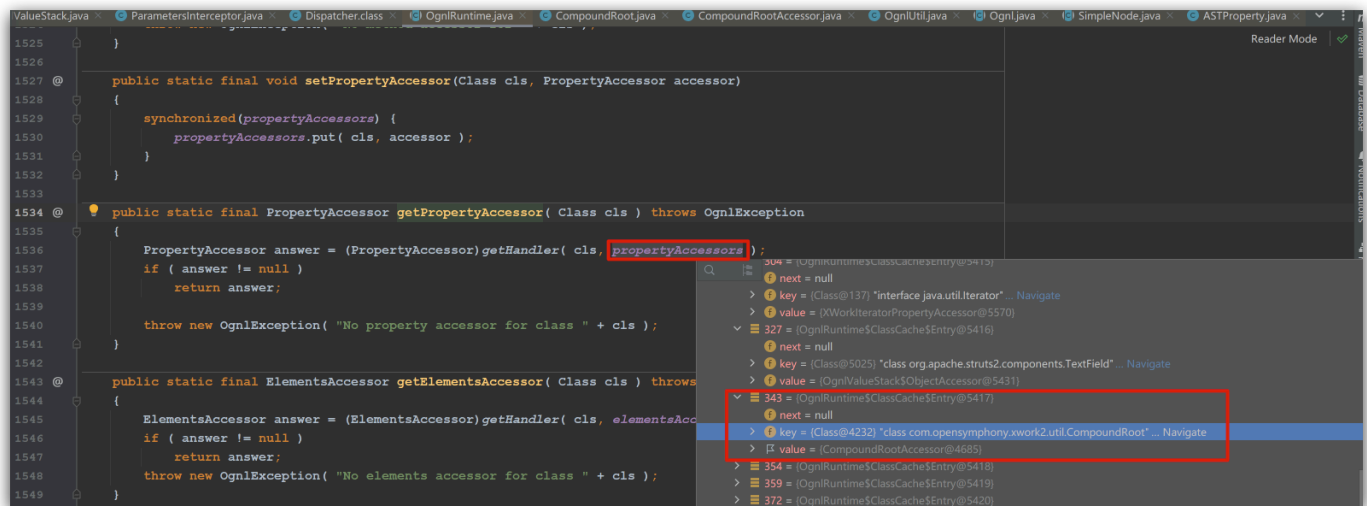
下图便是OGNL表达式在访问CompoundRoot时获取对应的CompoundRootAccessor



当然，在struts2框架中还有很多其它的Accessor，对应了其它的数据的访问方式，例如下图



所有的Accessor都在OgnlRuntime类中维护，但OgnlRuntime要处理的数据是各种各样的，所以要根据数据的类型找到对应的accessor去访问。访问OgnlValueStack（CompoundRoot）中的数据肯定是用CompoundRootAccessor，但如果访问来自其它地方的数据，就需要其他的accessor，如下图



2.0.8版本与2.2.1版本核心思想其实差异不大，对Ognl的计算、存储上变化也不是很大，但在安全性和包管理上有所区别

例如OgnlValue.setRoot函数，可以看出2.2.1版本增加了SecurityMemberAccess对象，这个类中有个属性allowStaticMethodAccess，如果为true，则允许OGNL调用静态函数

```
private void setRoot(CompoundRoot compoundRoot) {
    this.root = compoundRoot;
    this.context = Ognl.createDefaultContext(this.root, accessor, XWorkConverter.getInstance());
    context.put(VALUE_STACK, this);
    Ognl.setClassResolver(context, accessor);
    ((OgnlContext) context).setTraceEvaluations(false);
    ((OgnlContext) context).setKeepLastEvaluation(false);
}
```

```
protected void setRoot(XWorkConverter xworkConverter, CompoundRootAccessor accessor, CompoundRoot compoundRoot,
    boolean allowStaticMethodAccess) {
    this.root = compoundRoot;
    this.securityMemberAccess = new SecurityMemberAccess(allowStaticMethodAccess);
    this.context = Ognl.createDefaultContext(this.root, accessor, new OgnlTypeConverterWrapper(xworkConverter), securityMemberAccess);
    context.put(VALUE_STACK, this);
    Ognl.setClassResolver(context, accessor);
    ((OgnlContext) context).setTraceEvaluations(false);
    ((OgnlContext) context).setKeepLastEvaluation(false);
}
```

OgnlValueStackFactory的区别如下两个图，可以看出区别非常大，但Ognl计算、存储的思想没有本质变化

```

10     public class OgnlValueStackFactory extends ValueStackFactory {
11
12         @Override
13         public ValueStack createValueStack() {
14             return new OgnlValueStack();
15         }
16
17         @Override
18         public ValueStack createValueStack(ValueStack stack) {
19             return new OgnlValueStack(stack);
20         }
21
22     }

```

```

38     public class OgnlValueStackFactory implements ValueStackFactory {
39
40         private XWorkConverter xworkConverter;
41         private CompoundRootAccessor compoundRootAccessor;
42         private TextProvider textProvider;
43         private Container container;
44         private boolean allowStaticMethodAccess;
45
46         @Inject
47         public void setXWorkConverter(XWorkConverter conv) { this.xworkConverter = conv; }
48
49
50
51         @Inject("system")
52         public void setTextProvider(TextProvider textProvider) { this.textProvider = textProvider; }
53
54
55
56         @Inject(value="allowStaticMethodAccess", required=false)
57         public void setAllowStaticMethodAccess(String allowStaticMethodAccess) {
58             this.allowStaticMethodAccess = "true".equalsIgnoreCase(allowStaticMethodAccess);
59         }
60
61         public ValueStack createValueStack() {
62             ValueStack stack = new OgnlValueStack(xworkConverter, compoundRootAccessor, textProvider, allowStaticMethodAccess);
63             container.inject(stack);
64             stack.getContext().put(ActionContext.CONTAINER, container);
65             return stack;
66         }
67
68         public ValueStack createValueStack(ValueStack stack) {
69             ValueStack result = new OgnlValueStack(stack, xworkConverter, compoundRootAccessor, allowStaticMethodAccess);
70             container.inject(result);
71             stack.getContext().put(ActionContext.CONTAINER, container);
72             return result;
73         }

```

在分析漏洞过程中如果发现更多的区别，在继续补充。